FINAL “VIBE CODING” PROMPT (IMPROVED FOR YOUR DATA)

Copy this exactly

Build a full-stack web application called:

"Local Business Support Application with Integrated Healthcare Services"

This is a real-world, scalable project. The application must be designed in a way that works even with LIMITED DATA and can be upgraded later.

IMPORTANT DATA CONSTRAINT

Currently, I only have:

- Shop Name
- Locality (Area name)
- Some phone numbers
- Some images

I DO NOT have:

- Exact latitude/longitude
- Exact addresses
- Opening/closing time (for all shops)

So the system must:

1. Convert locality into approximate coordinates using geocoding (Map API)
2. Allow missing fields (optional data handling)
3. Be easily upgradable when more data is added later

CORE OBJECTIVE

- Show local businesses category-wise
- Allow users to browse nearby shops
- Sort shops based on approximate distance using locality-based coordinates
- Store all data in database
- Provide integrated healthcare module

□ TECH STACK

Frontend:

- React.js (responsive UI)
- Tailwind CSS

Backend:

- Node.js + Express.js

Database:

- MongoDB (Mongoose)

APIs:

- Use OpenStreetMap OR Google Maps Geocoding API to convert locality → coordinates

---

□ MAIN FEATURES

1. □ HOMEPAGE

- Category cards:
  - Grocery
  - Fashion
  - Healthcare
  - Services
  - Restaurants
- Search bar
- Optional: detect user location (if allowed)

---

2. □ CATEGORY SYSTEM
   Each shop must belong to:

- Grocery
- Fashion
- Healthcare
- Services
- Restaurants/Dhabas

---

3. □ LOCATION SYSTEM (IMPORTANT)
   Since exact coordinates are not available:

- Convert locality into latitude/longitude using API
- Store coordinates in database
- Calculate distance using Haversine formula
- Sort:
  Nearest → Farthest

---

4. □ SHOP LISTING
   Each shop must show:

- Name
- Image
- Locality
- Phone (if available)
- Distance (approximate)
- Status:
  - □ Open
  - □ Closed
    NOTE: If timing not available, show "Timing not available"

---

5. □ HEALTHCARE MODULE (SPECIAL)

- Separate route/page

Categories:

- Hospitals
- Clinics
- Pharmacies
- Diagnostic centers
- Optical stores

---

6. □ PHARMACY FEATURE

- Each pharmacy can have:
  - List of common medicines
  - Availability status
- If data missing, show placeholder

---

7. □ ADMIN PANEL
   Admin must be able to:

- Add shop
- Enter:
  - name
  - category
  - locality
  - phone
  - image
- System automatically:
  - converts locality → coordinates
  - stores in database

---

8. □ DATABASE DESIGN
   Collections:

Shops:

- name
- category
- locality
- latitude
- longitude
- phone (optional)
- image
- openingTime (optional)
- closingTime (optional)

Medicines:

- shopId
- medicineName
- availability

---

9. □ AUTHENTICATION

- Admin login using JWT

---

10. □ UI/UX

- Clean modern UI
- Card-based layout
- Mobile responsive

---

11. □ OUTPUT REQUIREMENTS

- Full frontend + backend code
- API routes
- Database schema
- Dummy data
- Instructions to run project

IMPORTANT:

- Handle missing data gracefully
- Do NOT assume perfect data
- Build scalable structure for future upgrades

HOW TO USE YOUR SAME PROMPT (STEP-BY-STEP VERSION)

I have **NOT changed your content**, only **broken it into stages** so you can paste gradually.

□ STEP 1 — PROJECT SETUP + FRONTEND BASE

Writing

Build a full-stack web application called:

"Local Business Support Application with Integrated Healthcare Services"

This is a real-world, scalable project. The application must be designed in a way that works even with LIMITED DATA and can be upgraded later.

□ IMPORTANT DATA CONSTRAINT

Currently, I only have:

- Shop Name
- Locality (Area name)
- Some phone numbers
- Some images

I DO NOT have:

- Exact latitude/longitude
- Exact addresses
- Opening/closing time (for all shops)

So the system must:

1. Convert locality into approximate coordinates using geocoding (Map API)
2. Allow missing fields (optional data handling)
3. Be easily upgradable when more data is added later

□ CORE OBJECTIVE

- Show local businesses category-wise
- Allow users to browse nearby shops

□ TECH STACK (FRONTEND ONLY FOR NOW)

Frontend:

- React.js (responsive UI)
- Tailwind CSS

□ MAIN FEATURES (FRONTEND PART)

1. □ HOMEPAGE

- Category cards:
  - Grocery
  - Fashion
  - Healthcare
  - Services
  - Restaurants
- Search bar
- Clean landing UI

---

2. □ CATEGORY SYSTEM

Each shop must belong to:

- Grocery
- Fashion
- Healthcare
- Services
- Restaurants/Dhabas

□ UI/UX

- Clean modern UI
- Card-based layout
- Mobile responsive

OUTPUT:

- Only frontend UI
- No backend yet

□ STEP 2 — SHOP LISTING + UI DETAILS

(After Step 1 is done)

Writing

Extend the existing project.

□ SHOP LISTING

Each shop must show:

- Name
- Image
- Locality
- Phone (if available)
- Status:
    - □ Open
    - □ Closed
      NOTE: If timing not available, show "Timing not available"

□ CATEGORY PAGES

- Clicking category → show shops under that category
- Use dummy data for now

□ SEARCH

- Search by shop name or category

IMPORTANT:

- Still frontend only
- Use mock/dummy data

□ STEP 3 — HEALTHCARE MODULE (FRONTEND)

Writing

Extend the project.

□ HEALTHCARE MODULE (SPECIAL)

- Separate route/page

Categories:

- Hospitals
- Clinics
- Pharmacies
- Diagnostic centers
- Optical stores

□ PHARMACY FEATURE

- Each pharmacy can have:
  - List of common medicines
  - Availability status

- If data missing, show placeholder

OUTPUT:

- Frontend healthcare pages only

⍰ STEP 4 — BACKEND SETUP

Writing

Now integrate backend.

□ TECH STACK (BACKEND)

Backend:

- Node.js + Express.js

Database:

- MongoDB (Mongoose)

□ DATABASE DESIGN

Shops:

- name
- category
- locality
- latitude
- longitude
- phone (optional)
- image
- openingTime (optional)
- closingTime (optional)

Medicines:

- shopId
- medicineName
- availability

□ ADMIN PANEL

Admin must be able to:

- Add shop
- Enter:
  - name
  - category
  - locality
  - phone
  - image

System automatically:

- converts locality → coordinates
- stores in database

OUTPUT:

- Backend APIs
- Database schema

□ STEP 5 — LOCATION + DISTANCE LOGIC

Writing

Extend backend.

□ LOCATION SYSTEM (IMPORTANT)

- Convert locality into latitude/longitude using API
- Store coordinates in database
- Calculate distance using Haversine formula
- Sort:
  Nearest → Farthest

APIs:

- Use OpenStreetMap OR Google Maps Geocoding API

⍰ STEP 6 — FRONTEND + BACKEND INTEGRATION

Writing

Connect frontend with backend.

□ FEATURES

- Fetch shops from API
- Replace dummy data with real database data
- Show distance dynamically

□ OUTPUT REQUIREMENTS

- Full frontend + backend connection
- API integration
- Instructions to run project

IMPORTANT:

- Handle missing data gracefully
- Do NOT assume perfect data
- Build scalable structure for future upgrades

⍰ STEP 7 — AUTHENTICATION

Writing

Add authentication.

□ AUTHENTICATION

- Admin login using JWT

- Secure routes

□ FINAL ANSWER TO YOUR QUESTION

□ Is breaking prompt into parts a good idea?

□ Not just good—**it's the correct professional approach.**

□ Benefits:

- Faster generation
- Better control
- Easier debugging
- Cleaner code
- You understand each part (important for viva)

□ One-shot prompt problems:

- Slow
- Confusing output
- Hard to fix errors
- Backend often broken

□ Pro Tip (important)

After each step:
□ Test it immediately
□ Don't move forward if something is broken